Neural Networks

Model

- neuron takes vector of inputs $x = [x_1, x_2, ..., x_n]^T$ and produces single output y
- every input x_i is multiplied by it's corresponding weight w_i, which gives **net activation**:

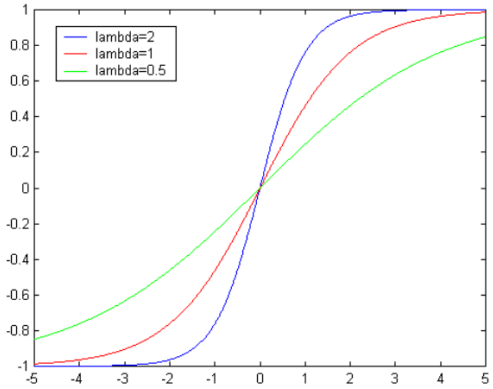
$$\text{net} = w^T x = \sum_{i=1}^{n} w_i x_i$$

- output is calculated by passing net activation through **activation function** f:

$$y = f(\text{net}) = f(w^T x)$$

  - two output convensions - **bipolar** $y \in [-1, 1]$ and **unipolar** $y \in [0, 1]$
  - two main function types
    - **discrete**: $f(\text{net}) = \text{sgn}(\text{net})$ - hard yes/no decision
    - **continiuous (sigmoid)** - with λ parameter:

$$f(\text{net}) = \frac{2}{1 + \exp(-\lambda \text{ net})} - 1$$



- $x_n = 1$ - **bias trick**, that enables us to treat w_n as bias for neuron, which simplifies notation

Learning

- **batch learning** - weights updated after one training epoch
- **incremental learning** - weights updated after each sample
- **supervised/unsupervised learning** - with or without target signal

Perceptron rule

- for supervised learning of **discrete** neurons:

$$r = d_i - y_i = d_i - \text{sgn}(w_i^t x)$$

$$\Delta w_i = c[d_i - \text{sgn}(w_i^T x)]x$$

  - if output is correct ($y_i = d_i$) then $r = 0 \rightarrow$ no change to weight
  - else ($r = \pm 2$) then weight is changed to correct error by $\pm 2cx_j$

Delta rule

- for supervised learning of **continiuous** neurons:

$$r = \delta = [d_i - f(w_i^T x)] f'(w_i^T x)$$

$$\Delta w_i = c[d_i - f(w_i^T x)] f'(w_i^T x) x$$

- using **derivative of the activation function** to update weight according to error gradient

One layer network

- J inputs $y_1, ..., y_J$ feed into K output neurons ($J \times K$ weights), that are producting outputs of $z_1, ..., z_K$
- **error function** - for a single sample y_l with target $d = [d_1, ..., d_K]^T$

$$E_l = \frac{1}{2} \sum_{k=1}^{K} (d_{lk} - z_{lk})^2$$

- we modify weights in the opposite direction to error gradient: $w^{k+1} = w^k - \eta \Delta E(w^k)$
  - η - learning rate parameter
- for bipolar sigmoidal function there is an elegant identity:

$$f'(\text{net}_k) = \frac{1}{2}(1 - z_k^2)$$

  - derivative calculated **only** from output
- sigmoidal function has a **vanishing gradient** problem - even if error $d - z$ is large, derivative at this point is near 0

Learning algorithm

- simple algorithm:
  1. Choose η (learning rate) and E_{max} (maximum acceptable error value)
  2. Initialize weights W
  3. Reset cumulative error $E_{\text{sum}} = 0$
  4. For each training sample:
     1. Compute the network output
     2. Add the sample's error to E_{sum}
     3. Update weights
  5. IF $E_{\text{sum}} < E_{\text{max}}$ stop, otherwise go to 3
- stopping criterion in simple algorithm is prone to overfitting, it's better to **stop early on a validation set**:
  5. Save previous validation error $E_{\text{val_ref}} = E_{\text{val}}$
  6. Compute classification error on validation set E_{val}
  7. If $E_{\text{val}} > E_{\text{val_ref}}$ stop, otherwise go to 3

Multi layer network

- able to represent **nonlinear decision boundries**
- I input neturons feed to J "hidden" neurons, which feed to K output neurons
- problem - hidden neurons don't have target values
- sollution - **backpropagation**
  - error signal signal of hidden neuron is a weighted sum of the deltas of the neurons it feeds into, multipled by it's own derivative
  - we calculate error signals at the last layer before output from target values, and then propagate it back:

$$\Delta v_{ij} = \eta \left(\sum_{k=1}^{K} \delta_{zk} w_{k_j} \right) f_{j'}(\text{net}_j) x_i$$

Learning algorithm

1. Choose η (learning rate) and $E_{\max}$ (maximum acceptable error value)
2. Initialize W and V (weights between all layers)
3. Reset E_{sum}
4. For each training sample:
    1. Compute the networks output
    2. Add the sample's error to E_{sum}
    3. Compute error signal vectors
    4. Update weights to all layers
5. Check stopping (again, preferably on validation set)

Learning parameters tips

- update timing
    - incremental - after each sample
    - batch - update once per epoch, you sum Δw across all samples and then perform update
- escaping local minimas
    - adding random perturbations to the weights
    - adding noise to inputs
    - randomizing order of training samples each epoch
- initial weights
    - trying multiple random initializations and keeping the best
    - if weights are too large, neurons saturate, $f' = 0$, so learning stops
    - if weights are all 0, output is undefined
    - general rule - set random weights so that total net activation is about 1
- learning rate
    - too small - slow learning, trapping in local minimas
    - too large - oscilations
    - solution - variable learning rate, increasing by constant value if error systematically decreases, decreasing geometrically when error increases